COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/22/2026	**DATE SUBMITTED:**	7/22/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 3.1 Singly Linked List

1. PROCEDURE OUTPUT

main.cpp

main.cpp > ...

```cpp
#include <iostream>
#include "singly_ll.h"

int main(){

    //create nodes:
    SinglyList<char>* head = nullptr;
    SinglyList<char>* second = nullptr;
    SinglyList<char>* third = nullptr;
    SinglyList<char>* fourth = nullptr;

    //allocate memory to the head
    head = new SinglyList<char>;
    second = new SinglyList<char>;
    third = new SinglyList<char>;
    fourth = new SinglyList<char>;

    //added the data to the node
    head->data = 'R';
    second->data = 'A';
    third->data = 'P';
    fourth->data = 'H';

    //assign the next to the node
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = nullptr;

    std::cout << "Testing the ListTraversal Fucntion: \n";
    ListTraversal(head);

    std::cout << "Testing the Insert at Head Fucntion: \n";
    sllInsertHead('I', &head);
    ListTraversal(head);

    std::cout << "Testing the Insert any location Function: \n";
    sllGeneralInsert('X', head->next->next);
    ListTraversal(head);

    std::cout << "Testing the Insert at end Function: \n";
    sllInsertEnd('U', &head);
    ListTraversal(head);

    std::cout << "Testing the delete a nodeFunction: \n";
    sllDeleteNode('X', &head);
    ListTraversal(head);
```

```
48      sllDeleteNode('Q', &head);
49      ListTraversal(head);
50      sllDeleteNode('U', &head);
51      ListTraversal(head);
52
53      std::cout<<"Deleteing the whole list: \n";
54      sllDeleteList(&head);
55      ListTraversal(head);
56
57      return 0;
58  }
59
```

singly_ll.h

singly_ll.h

singly_ll.h > ListTraversal<T>(SinglyList<T>*)

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template <typename T>
class SinglyList{
    public:
    T data;
    SinglyList<T>* next;
};

template <typename T>
void ListTraversal(SinglyList<T>* node){

    if(node == nullptr){
        std::cout << "the list is empty" << std::endl;
        return;
    }

    //WHILE n IS NOT EQUAL TO null:
    while(node != nullptr){
        //PRINT data to n
        std::cout << node->data;

        if(node->next !=nullptr){
        std::cout << "->";
        }
            //GO TO THE NEXT NODE:
        node = node->next;

    }
    //PRINT THE NEXT LINE
    std::cout << std::endl;
}

template <typename T>
void sllInsertHead(T newData, SinglyList<T>** currentHead){

    // 1. Allocate memeory for then new node
    SinglyList<T>* newNode = new SinglyList<T>;

    // 2. Put out data into the new node
    newNode->data = newData;

    // 3. Set Next of the new node to point ot the previous head
    newNode->next = *currentHead;

    // 4. Reset Head to point to the new nod
```

```
48      *currentHead = newNode;
49
50  }
51
52  template <typename T>
53  void sllGeneralInsert(T newData, SinglyList<T>* prevNode){
54
55      // 1. Check if it is the head node
56      if(prevNode == nullptr){
57      // 2. If null, print "Previous node cannot be null."
58          std::cout << "Previous node cannot be null." << std::endl;
59      }
60      // 3. Allocate a new node
61      SinglyList<T>* newNode = new SinglyList<T>;
62      // 4. Store data in the new node
63      newNode->data = newData;
64      // 5. Point new node to the node previous node was pointing to
65      newNode->next = prevNode->next;
66      // 6. Point previous node to the new node
67      prevNode->next = newNode;
68  }
69
70  template <typename T>
71  void sllInsertEnd(T newData, SinglyList<T>** head){
72      // 1. Allocate new node
73      SinglyList<T>* newNode = new SinglyList<T>;
74      //dereference to the new node
75      SinglyList<T>* currentNode = *head;
76      //store data in new node:
77      newNode->data = newData;
78      //point the new node to null
79      newNode->next = nullptr;
80
81      //Traverse the list of the current node to the new node
82      while(currentNode->next != nullptr){
83          currentNode = currentNode->next;
84      }
85
86      //Point the next of the current node to the new node
87      currentNode->next = newNode;
88
89  }
90
91  template <typename T>
92  void sllDeleteNode(T findData, SinglyList<T>** head){
```

```cpp
void sllDeleteNode(T findData, SinglyList<T>** head){
    //check first if the list is already empty??
    if(*head == nullptr) return;

    SinglyList<T>* currNode= *head;
    SinglyList<T>* prevNode= nullptr;

    //search for the node 1. what if the data we are looking for is not... (idk)
    // 2. the data is found
    while(currNode != nullptr && currNode->data != findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }

    //data was not found
    if(currNode == nullptr) return;

    if(prevNode == nullptr){
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}

template <typename T>
void sllDeleteList(SinglyList<T>** head){

    SinglyList<T>* current = *head;

    while(current != nullptr){
        SinglyList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;
}

#endif
```

OUTPUT:

```
Testing the ListTraversal Fucntion:
R->A->P->H
Testing the Insert at Head Fucntion:
I->R->A->P->H
Testing the Insert any location Function:
I->R->A->X->P->H
Testing the Insert at end Function:
I->R->A->X->P->H->U
Testing the delete a nodeFunction:
I->R->A->P->H->U
I->R->A->P->H->U
I->R->A->P->H
Deleteing the whole list:
the list is empty
```

Reflection:

As a result of this activity, I learned more about working with singly linked lists from the lecture. I managed to understand the first half of the lecture on making nodes, allocating memory, using new to make the linked list, placing data into the nodes, and linking them using pointers. Nevertheless, half way of trying to learn the code turned out to be more complicated due to using parameters of type pointer-to-pointer such as ** in functions and operations with the head node. However, watching the execution of code in the lecture helped me understand how all functions worked together in order to make and operate on the linked list. I just need to understand the logic behind it more for me to make my own linklist when doing another activity.

This activity made me understand the importance of linked lists. In contrast to arrays, linked lists do not require contiguous memory allocation and may dynamically change in size thanks to modification of pointers. In that way, operations of insertion and deletion become much easier in some cases. Nevertheless, arrays are very convenient since they have elements that may be accessed directly using an index, but using a linked list and modifying it with pointers seems more efficient; it just needs more logical understanding. Even though I still need more practice with pointers and the more advanced functions from the singly_ll.h, I have an idea of what it is trying to do just by looking at its output. I also understand what is trying to do when it tries to replace one of the nodes with new data, I just have to practice, analyze the code, and learn how to use the right syntax and attributes for me to have a more advanced understanding.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **ChatGPT and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar, Code explanation and concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI to explain the concepts step by step for me to be able to have a better understanding and used QuillBot to help with grammar related work. All the code was from the lecture and some lines of code were modified by me.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.